CH02 HTML结构与语义

一个页面能够显示文字，并不意味着它已经把内容关系表达清楚。活动名称是标题还是普通段落？一张图片在说明什么？日程表中的某个时间对应哪项内容？输入框旁的文字是否真正成为控件的名称？这些问题需要由HTML结构回答。

本章以校园活动信息页为线索，把元素、属性和嵌套关系连接起来，进一步组织页面区域、资源、表格与静态表单。判断结构是否合理，需要同时阅读源代码、检查浏览器形成的节点关系，并验证链接、图片和原生控件的实际行为。

1 元素、属性与文档树

1.1 从一个链接读出元素的组成

```
<a class="more-link" href="pages/details.html">查看活动详情</a>
```

`a`是元素名，`<a ...>`是开始标签，`</a>`是结束标签，中间的文字是内容。`class`和`href`是开始标签中的属性。`href`指定链接目标，`class`为元素提供一个或多个类名，用于分组和后续处理。



图2-1 从完整链接读出标签、属性与内容

标签是源代码中的标记，元素是文档中的一个结构单位。两者在口语中经常混用，阅读代码时应区分：修改链接文字影响内容，修改`href`影响目标，修改元素名可能改变角色和浏览器行为。

1.2 成对元素、嵌套与空元素

普通成对元素可以包含文本，也可能包含允许的子元素：

```
<p>请带来一本<strong>自己读过的书</strong>。</p>
```

外层`p`包含文字与`strong`。关闭时应先结束内层，再结束外层。为了便于检查，可以把较长的嵌套关系分行缩进，但缩进本身不会建立父子关系。

有些元素是空元素，不允许包含子内容，也没有结束标签。例如：

```
<img src="images/activity.svg" alt="展开的书与交流气泡">
<input id="nickname" name="nickname" type="text">
<br>
```

不能为`img`补写一对内容标签，也不能把图片说明放进`<img>...</img>`中。`<img ... />`中的末尾斜杠在HTML中不会把普通元素变成空元素；不能把`<div />`当作已经关闭的`div`。空元素的身份来自HTML规则。



图2-2 看清三种写法：成对、嵌套与空元素

1.3 全局属性与元素专用属性

全局属性可以用于多种HTML元素。常见的包括：

属性	作用	例子
id	在当前文档中唯一标识一个元素	`id="schedule"`
class	为元素提供一个或多个类名	`class="activity-card notice-card"`
lang	说明内容的主要语言	`lang="zh-CN"`
title	提供补充咨询信息	`title="查看活动详情"`
hidden	表示内容当前不相关，应隐藏	`hidden`

同一文档中的`id`应唯一。`class`可以重复使用，一个元素也可以具有多个类，多个类名用空格分隔。给元素起名并不会自动产生功能；例如`class="button"`不会把`div`变成按钮。

`href`主要用于链接等特定元素，`src`用于引用相应资源，`name`和`value`在表单控件中有特定含义。学习属性时应把“属性用于谁”和“它表达什么”一起检查，而不是把所有属性随意放到任意标签上。

`title`不适合承载用户必须获得的信息。补充提示未必在触屏或键盘操作中同样可用，关键说明应直接放在页面中。

1.4 布尔属性看是否存在

`required`、`disabled`和`checked`是布尔属性的例子。对于这些属性，写出属性就表示相应状态为真，省略才表示假。

```
<input id="nickname" name="nickname" type="text" required>
<button type="submit" disabled>暂不提交</button>
<input type="checkbox" name="reminder" value="yes" checked>
```

下面的代码仍然要求填写输入框：

```
<input name="nickname" required="false">
```

因为`required`依然存在，字符串`"false"`并不会将它关闭。需要取消必填时，应移除整个属性。并非所有“看起来表示状态”的属性都是布尔属性；例如`aria-hidden`采用字符串值规则，不能把两者混为一谈。



图2-3 属性有不同规则：标识、分组与状态

1.5 嵌套形成父子、祖先和后代关系

```
<main>
  <section>
    <h2>活动介绍</h2>
    <p>带一本喜欢的书参加交流。</p>
  </section>
</main>
```

`section`是`main`的子元素，`h2`和`p`是`section`的子元素。`main`是后两者的祖先，但不是它们的直接父元素。`h2`与`p`具有同一个父元素，因此是兄弟元素。

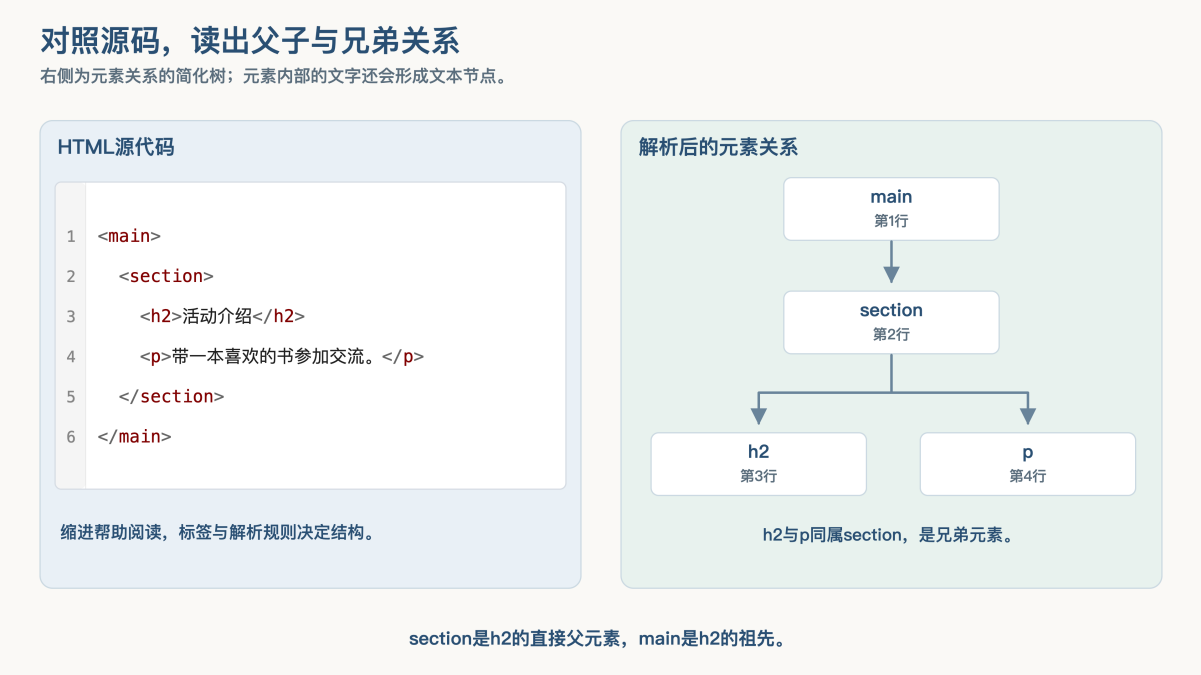


图2-4 对照源码，读出父子与兄弟关系

浏览器把HTML解析为DOM（Document Object Model，文档对象模型）树。树中除了元素节点，还有文本等其他节点。讨论“两个子元素”时，关注的是元素关系；计算所有子节点时，还可能涉及文字和空白文本。后续选择元素与操作页面时，这一区分会再次出现。

源代码缩进清楚，有助于人看懂树；真正的结构来自标签和解析规则，而不是每行前面的空格数量。

1.6 合法嵌套与浏览器容错

元素允许包含的内容由HTML规则限定。普通列表应把项目放在`li`中，`p`不能包围任意分区结构，交互元素也不能随意互相嵌套。例如，不应把按钮放入带`href`的链接中。

```
<!-- 不合理：链接内部又包含按钮。 -->
<a href="pages/details.html">
  <button type="button">查看详情</button>
</a>
```

查看详情是导航操作，直接使用链接即可：

```
<a href="pages/details.html">查看详情</a>
```

浏览器会对某些错误进行恢复。例如：

```
<p>活动介绍<div>补充说明</div></p>
```

遇到`div`开始标签时，浏览器会先结束前面的段落。解析后的元素树并不是“p包含div”，还可能出现由末尾`</p>`触发的空段落。浏览器保护用户继续阅读的能力，不代表源代码正确。



图2-5 源码中的嵌套意图，不一定成为实际结构

用“查看源代码”检查文件写法，用Elements检查解析结果。发生差异时，应修正源代码，不能仅因页面看起来还能显示就保留错误结构。

2 让页面区域表达明确的含义

2.1 先确定主题，再划分区域

校园活动信息页通常包括活动名称、页面导航、活动介绍、日程、参与要求和页尾信息。语义化结构的第一步，是判断这些内容承担什么角色，再选择相应元素。

元素	主要角色	常见使用位置
header	一份页面或某个区域的介绍性内容	页面标题与介绍
nav	主要导航链接区域	前往活动介绍、日程等位置的链接
main	文档的主要内容	当前页面的核心信息
section	围绕一个主题组织的内容区域	活动介绍、活动安排
article	可以独立理解或分发的完整内容	一篇文章、一则独立活动报道
aside	与主要内容相关但可分离的补充内容	补充阅读、相关说明
footer	页面或区域的页尾信息	署名、相关链接与版权说明

`header`和`footer`可以出现在不同区域中，不只对应屏幕顶部与底部。`section`和`article`也不应根据“想要一个矩形容器”来选择，而应根据内容关系选择。



图2-6 选元素，先问这部分内容承担什么角色

2.2 标题层级与页面骨架共同组织内容

```
<header>
  <h1>校园读书交流会</h1>
  <nav aria-label="页面导航">
    <a href="#introduction">活动介绍</a>
    <a href="#schedule">活动安排</a>
  </nav>
</header>
<main>
```

```
  <section id="introduction">
    <h2>活动介绍</h2>
    <p>带一本喜欢的书，分享一次阅读体验。</p>
  </section>
  <section id="schedule">
    <h2>活动安排</h2>
    <p>周五下午在图书馆交流空间举行。</p>
  </section>
</main>
<footer>
  <p>读书交流社 · 活动示例</p>
</footer>
```

导航中的片段标识符`#introduction`与目标区域的`id`对应。每个`section`围绕一个主题，`h2`说明主题是什么。仅阅读标题，便能还原页面的主要线索。

`aria-label`为导航区域提供名称。合适的原生HTML应先写清楚，再按实际需要补充属性；不能用大量ARIA属性代替合理的元素选择。



图2-7 页面区域表达角色，标题说明各部分主题

2.3 通用容器div与span

`div`和`span`是没有特定内容角色的通用容器。`div`常用来把若干元素分为一组，`span`常用来包围一段文本中的局部内容。

```
<div class="form-field">
  <label for="nickname">练习昵称</label>
  <input id="nickname" name="nickname" type="text">
</div>
<p>报名状态：<span class="status-text">尚未开放</span></p>
```

上面的`div`表示一组标签与控件，不需要为这一包装层硬造一个独立主题。`span`标记局部文字，也不会把它变成标题或按钮。

当内容是导航时用`nav`，当内容是按钮时用`button`，当内容只是需要分组时才使用通用容器。语义元素不是越多越好，准确比数量更重要。



图2-8 div分组一块内容，span标记局部文字

2.4 内容角色与默认外观

浏览器默认把标题显示得较大，把段落分开，把链接显示为可识别的文字。这些外观有助于阅读，但不能反过来用视觉大小决定元素身份。

一个`h1`表达最高层级标题，一个`p`表达段落。即使后续调整了字号，二者的含义也不会互换。通常默认另起一行的元素与通常随文字排列的元素，提供了不同的阅读效果；它们的显示行为将在CH03讨论，而不是HTML嵌套合法性的最终依据。

不能仅用“块级元素”和“行内元素”两类概括全部HTML规则。例如，一个链接允许包含什么，既与内容模型有关，也受到交互内容限制。需要判断具体嵌套时，应查阅对应元素的定义。



图2-9 看起来多大，与它是什么角色，是两个问题

3 用地址关联页面、图片与位置

3.1 当前文档是相对路径的计算起点

采用下面的项目目录：

```
campus-event/
├── index.html
├── images/
│   └── activity.svg
└── pages/
    └── details.html
```

在未设置`base`元素的这些示例中，相对地址以当前文档URL为基准解析。在`index.html`中可以这样引用：

```
<a href="pages/details.html">阅读活动详情</a>
<img src="images/activity.svg" alt="展开的书与交流气泡">
```

在`pages/details.html`中，当前目录层次发生变化，需要先返回上一级：

```
<a href="../index.html">返回活动首页</a>
<img src="../images/activity.svg" alt="展开的书与交流气泡">
```

`../`表示上一级路径，`./`表示当前路径。资源文件没有移动，同一资源从不同文档引用时，写法仍可能不同。



图2-10 同一个资源，从不同文档出发写不同路径

3.2 相对地址、绝对URL与站点根路径

地址形式	例子	解释
相对地址	`images/activity.svg`	根据当前文档位置解析
绝对URL	`https://example.com/news.html`	包含协议方案与主机等信息
从站点根开始的路径	`/images/activity.svg`	在HTTP站点中从当前站点根路径解析
页内片段	`#schedule`	指向当前文档中对应id的位置

`/images/activity.svg`不是“当前项目文件夹中的images”。在本地`file:`访问中，前导斜杠通常会指向文件系统根等位置，不能假定它与站点部署效果一致。初学项目使用相对路径，更容易直接检查文件关系。

不要把自己计算机上的绝对磁盘路径复制进页面，例如某个用户目录中的图片地址。其他人的目录与自己的不同，移动项目后也容易失效。应把需要的资源放进项目，再使用项目内部的相对地址。

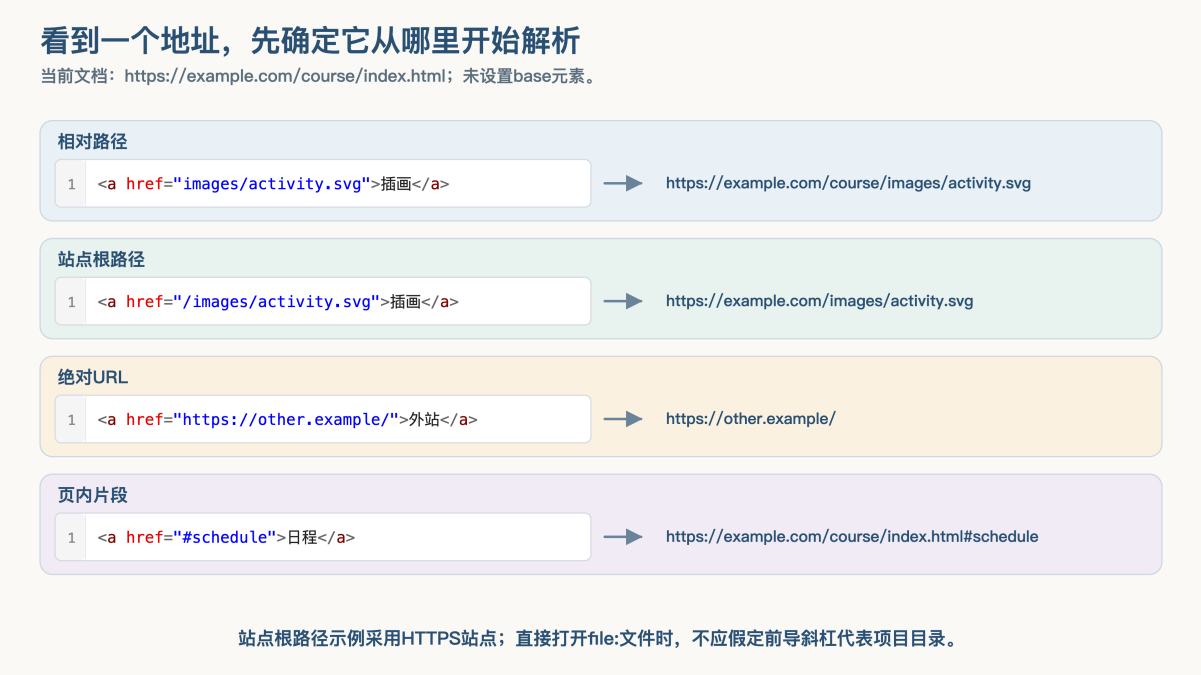


图2-11 看到一个地址，先确定它从哪里开始解析

3.3 页内导航与唯一id

```
<a href="#registration">查看参与方式</a>
<section id="registration">
  <h2>参与方式</h2>
  <p>阅读说明后准备交流内容。</p>
</section>
```

片段中的名字应与目标`id`一致。`id`不包含开头的`#`，`href`中的`#`表示片段。重复使用相同`id`会使目标不明确，也会影响标签关联与后续代码操作。

跨文档也可以同时指定文件与片段：

```
<a href="../index.html#schedule">返回首页查看日程</a>
```

检查链接时，可以先算出目标文档，再核对目标文档中是否存在相应`id`，避免把文件路径问题与片段问题混在一起。



图2-12 href中的片段，怎样找到目标区域

3.4 图片、替代文本与图注

```
<figure>
  <img src="images/activity.svg"
       alt="展开的书与交流气泡的插画" width="800" height="300">
  <figcaption>从一本书开始，让不同的想法在交流中相遇。</figcaption>
</figure>
```

`src`指定资源，`alt`提供图片的文字替代，`figure`组织图片及相关内容，`figcaption`提供可见图注。`alt`与图注解决的问题不同，不需要机械重复同一句话。

图片传达了内容时，替代文本应表达它在当前语境中的信息。若图片是活动海报而重要时间只写在海报里，替代文本或邻近正文也需要让用户获取这些信息。纯装饰、且不承担内容功能的图片可以使用空替代文本`alt=""`。

`width`和`height`可声明图片尺寸及其比例信息，帮助浏览器预留空间。它们不是图片语义的一部分，也不能替代资源路径与替代文本。图片无法显示时，先检查文件存在、路径和文件名大小写，再检查尺寸。



图2-13 图片在哪里、表达什么、图注说什么

3.5 让链接目的清楚可辨

“阅读活动详情”“返回活动首页”“查看交流流程”比一组相同的“点击这里”更明确。链接文字应与目标内容相符，不应把任意地址包装成一个看似可信的名称。

通常让链接在当前标签页打开即可。若确有需要使用`target="_blank"`，应让用户知道会打开新标签页，并可配合`rel="noopener"`说明与新窗口的关系。不能仅通过链接文字推断地址安全或内容真实，测试时仍应核对实际目标。



图2-14 链接既要能跳转，也要让人知道去哪里

4 用表格表达行列关系

4.1 先判断内容是不是二维数据

活动日程同时具有“时间”和“内容”两个维度，适合使用表格。文章段落、侧边栏与页头的摆放则不是二维数据，不应借表格来拼页面布局。

下面的表格有一行列标题和两行数据：

```
<table>
  <caption>读书交流会流程</caption>
  <thead>
    <tr><th scope="col">时间</th><th scope="col">内容</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">14:00</th><td>相互认识</td></tr>
    <tr><th scope="row">14:10</th><td>分享阅读体验</td></tr>
  </tbody>
</table>
```



图2-15 把表格源码映射到行、列与标题

`table`建立表格，`tr`建立一行，`th`是标题单元格，`td`是数据单元格。`thead`和`tbody`组织标题与主体数据。`caption`说明整张表格表达什么。

4.2 标题单元格提供阅读方向

`scope="col"`说明标题适用于一列，`scope="row"`说明标题适用于一行。在例子中，“内容”是列标题，“14:10”是对应行的时间标题。用户读取“分享阅读体验”时，可以据此理解它与哪一时间及哪一类信息关联。

用加粗文字代替`th`，或把表名写在一个外观相似的普通段落中，不能建立相同的表格关系。复杂表格需要更明确的标题关联，初学时应优先保持简单的行列结构。



图2-16 读一个数据单元格，要同时找到行标题和列标题

4.3 colspan与rowspan覆盖网格位置

普通单元格默认覆盖一个网格位置。`colspan`向多个列位置延伸，`rowspan`向多个行位置延伸。已经被覆盖的位置不再写入一个重复单元格。

```
<table>
  <caption>两日交流安排</caption>
  <thead>
    <tr>
      <th scope="col">日期</th>
      <th scope="col">上午</th>
      <th scope="col">下午</th>
```

```
    </tr>
  </thead>
  <tbody>
    <tr><th scope="row">周五</th><td>阅读分享</td><td>书目交流</td>
</tr>
    <tr><th scope="row">周六</th><td colspan="2">主题展览</td></tr>
  </tbody>
</table>
```

最后一行只有两个单元格元素，但覆盖三个列位置。`主题展览`覆盖上午与下午两列，不能再额外添加一个空`td`来“补齐标签数”。

同样地，下面两行的“图书馆”共享一个跨两行单元格：

```
<tbody>
  <tr><th scope="row">第一组</th><td rowspan="2">图书馆</td>
<td>14:00</td></tr>
  <tr><th scope="row">第二组</th><td>15:00</td></tr>
</tbody>
```

第二行只需为第一列与第三列写单元格，因为第二列已经被上一行覆盖。跨行单元格应留在同一行组内，不能随意跨越`thead`与`tbody`等分组边界。

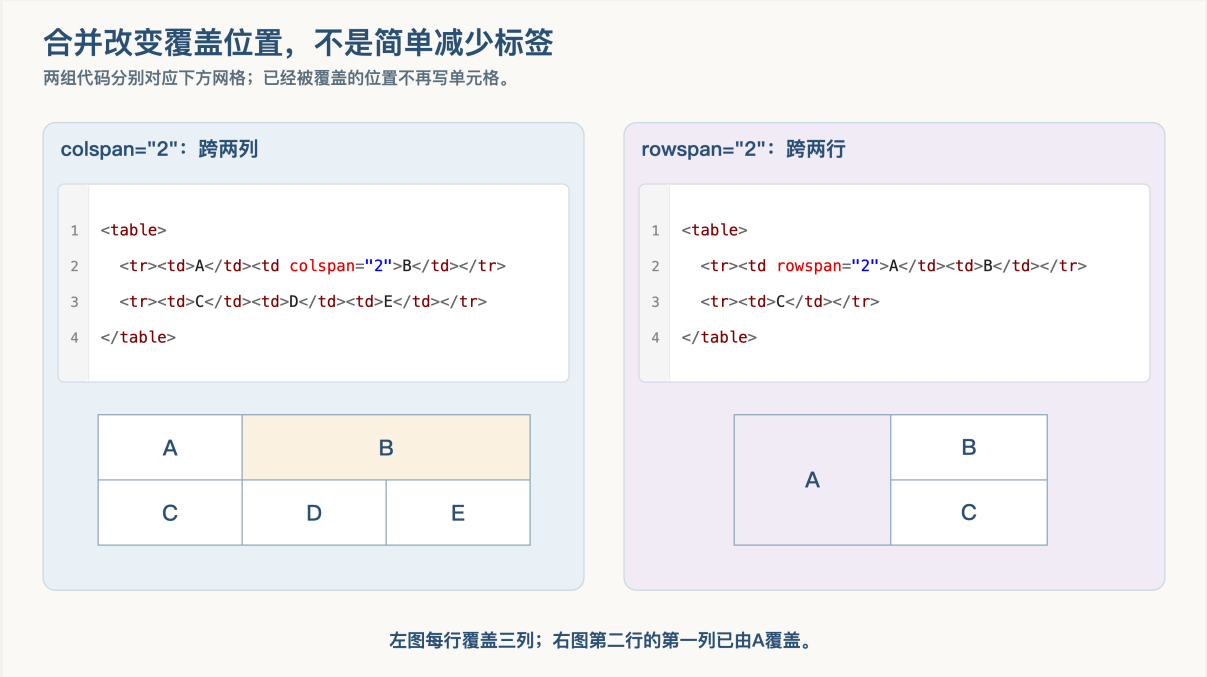


图2-17 合并改变覆盖位置，不是简单减少标签

4.4 通过网格检查而不是只数标签

检查表格时，先画出期望的行列，再把每个单元格对应到覆盖的位置。每一行都应与同一张网格一致，不能覆盖重叠，也不能因为遗漏位置而产生意外空缺。

没有编写边框规则时，浏览器中的表格可能没有明显网格线，这不表示它没有行列结构。可以在Elements中检查`tr`及单元格关系。合并越复杂，阅读与标题关联越困难；能用简单表格清楚表达时，就不增加合并。

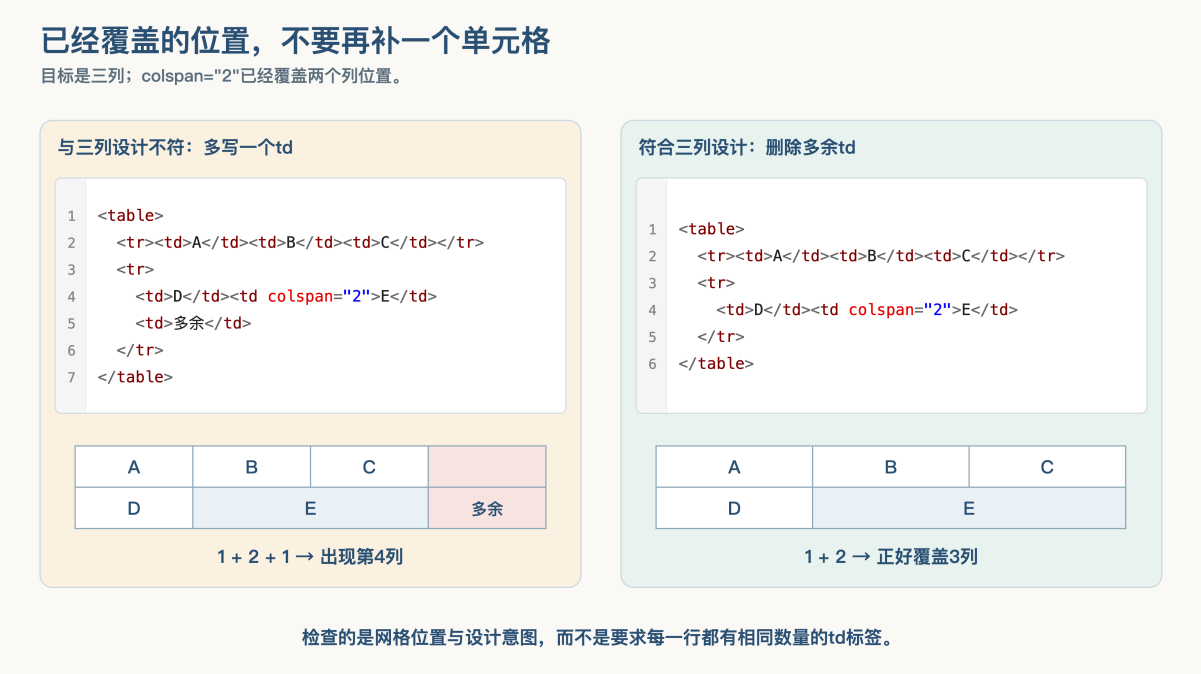


图2-18 已经覆盖的位置，不要再补一个单元格

5 用表单组织输入与字段

5.1 表单既包含控件，也表达数据

`form`组织一组可以提交的控件。`input`提供单行或特定类型输入，`textarea`提供多行输入，`select`与`option`提供选项，`button`表示按钮。

```
<form>
  <p>
    <label for="nickname">练习昵称</label>
    <input id="nickname" name="nickname" type="text" required>
  </p>
  <p>
    <label for="book-type">书籍类别</label>
    <select id="book-type" name="bookType">
```

```
      <option value="fiction">文学与小说</option>
      <option value="science">科学与技术</option>
    </select>
  </p>
  <button type="submit" disabled>静态练习，暂不提交</button>
</form>
```

这里可以观察输入与选择，但提交按钮被禁用。输入框中出现了文字，只说明浏览器中的当前控件值发生变化，并不表示服务器接收或保存了报名记录。页面主例用于练习结构；另设的本地提交示例用于观察地址变化。

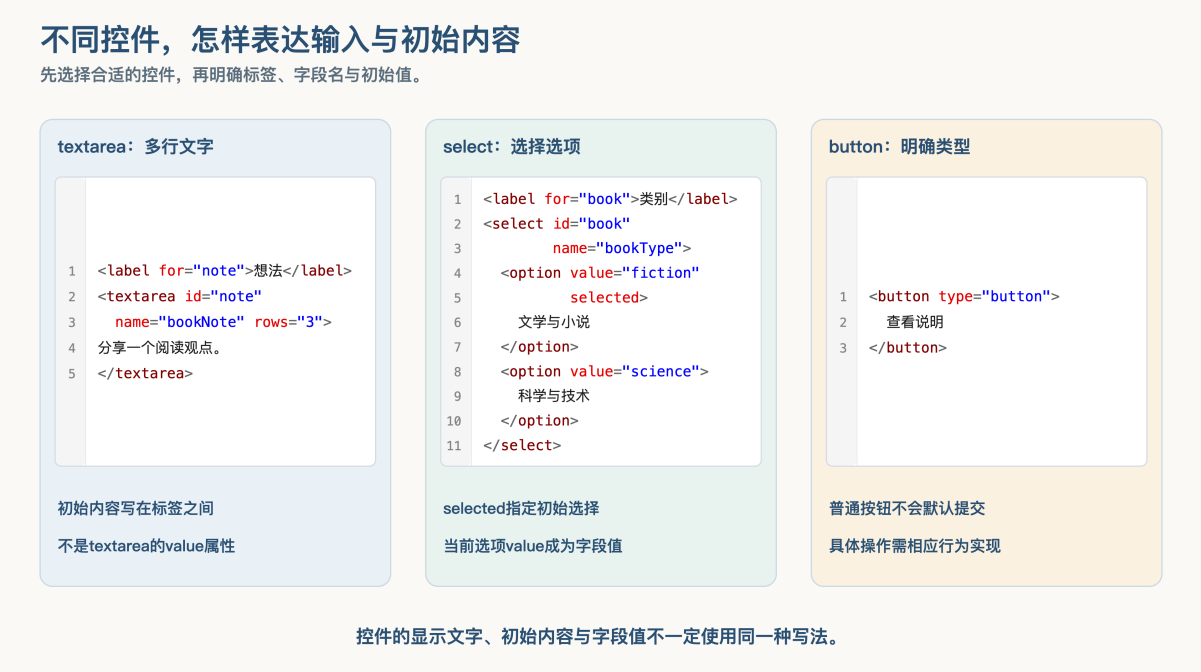


图2-19 不同控件，怎样表达输入与初始内容

5.2 label与控件之间的明确关联

```
<label for="nickname">练习昵称</label>
<input id="nickname" name="nickname" type="text">
```

`for`与控件`id`一致，建立标签关系。点击标签可以把焦点移到相应输入框，辅助技术也可以据此识别控件名称。只把一段文字视觉上放在输入框旁边，不能保证建立这种关系。

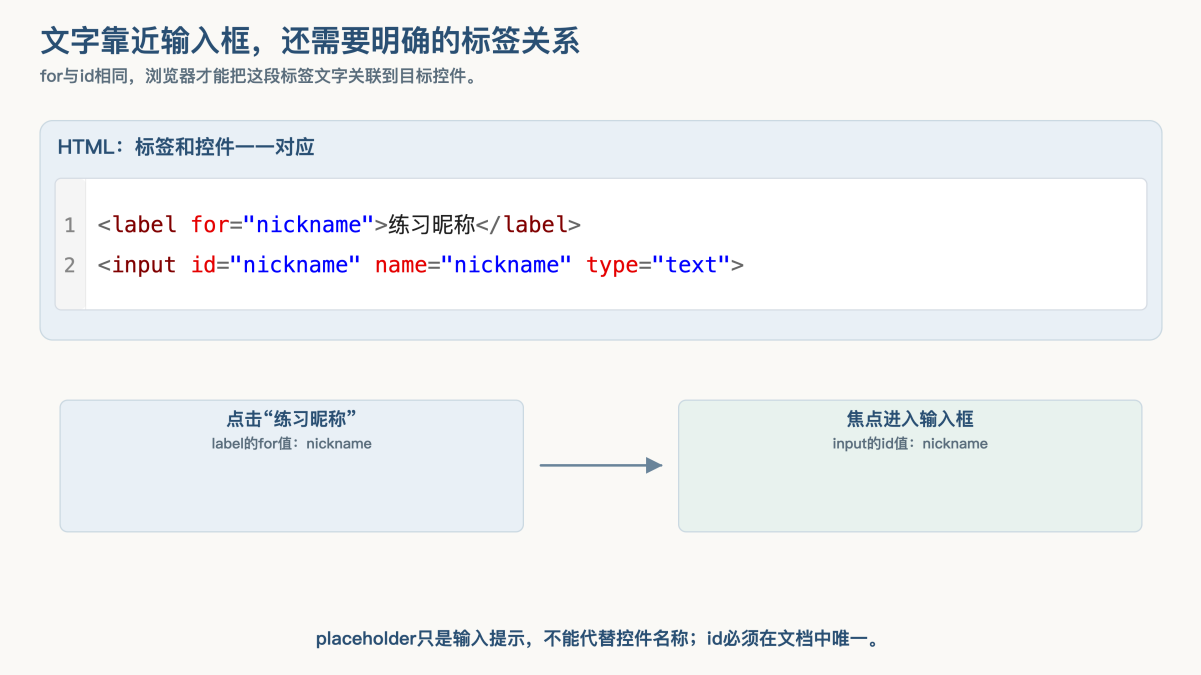


图2-20 文字靠近输入框，还需要明确的标签关系

另一种写法是把控件直接包含在`label`内部：

```
<label>
  <input type="checkbox" name="reminder" value="yes">
  愿意接收活动提醒
</label>
```

`placeholder`提供暂时的输入提示，输入后会消失，不能代替始终能够识别的标签。对`select`和`textarea`也应提供明确名称。

5.3 id、name与value的不同职责

```
<input id="nickname" name="nickname" type="text" value="小南">
```

`id`在文档中标识元素，可以用于标签关联和页内定位；`name`给提交字段命名；初始`value`给出控件开始时的值。用户编辑后，提交使用当前控件值，不是始终使用HTML属性里的原始文字。



图2-21 id用于文档定位，name与当前值形成字段

若昵称当前是“小南”，可以把数据理解成`nickname=小南`。对于选择框：

```
<select id="book-type" name="bookType">
  <option value="fiction">文学与小说</option>
  <option value="science">科学与技术</option>
</select>
```

用户看到“科学与技术”，但相应字段值是`science`。显示文字与提交值并不一定相同。只有`id`、没有`name`的普通输入框，不会因此自动成为一个命名提交字段。

5.4 单选、多选与控件分组

同一表单中，相同`name`的单选按钮可以组成一个选择组：

```
<fieldset>
  <legend>参与方式</legend>
  <label>
    <input type="radio" name="role" value="listener" checked>
    旁听交流
  </label>
  <label>
    <input type="radio" name="role" value="speaker">
    分享一本书
  </label>
</fieldset>
```

选择“分享一本书”后，当前字段可以表达为`role=speaker`。`fieldset`组织相关控件，`legend`为这组控件提供说明。单选适合互斥选项，复选框适合独立开关或多选。未选中的单选与复选控件通常不会产生对应提交项。

`checked`表示初始选中状态，之后用户可以改变选择；它不意味着选项永远不可更改。`disabled`则使控件不可操作，被禁用的控件也不会参与正常表单数据提交。



图2-22 相同name的单选互斥，复选项可以独立选择

5.5 输入类型与原生校验

TYPE或属性	用途	应注意的边界
text	普通单行文字	不自动判断业务含义
email	邮箱输入与基础格式检查	格式通过不代表邮箱真实存在
number	数值输入	可结合min、max与step限制范围
date	日期输入	控件界面可能因浏览器而不同
password	遮蔽显示输入字符	不等于加密传输或安全存储
required	对支持的控件要求填写	空值限制不等于内容真实性检查

`textarea`的初始内容写在开始与结束标签之间，而不是写成`value`属性。`select`通过选中的`option`表达当前选择，`selected`可指定选项的初始状态。

原生校验帮助用户及早发现常见输入问题，但不能代替服务器检查。客户端规则可以被修改或绕过，真实应用必须按自己的业务与安全要求处理收到的数据。

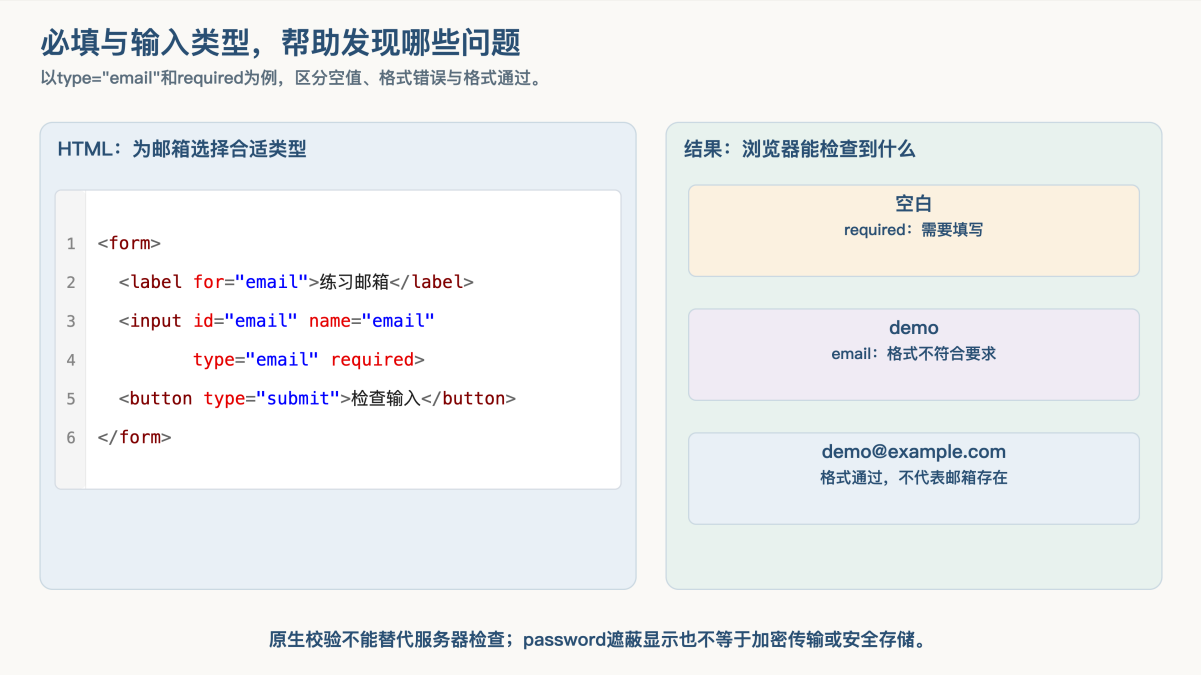


图2-23 必填与输入类型，帮助发现哪些问题

5.6 提交目的地与按钮类型

```
<form action="pages/received.html" method="get">
  <label for="practice-nickname">练习昵称</label>
  <input id="practice-nickname" name="nickname" type="text"
required>
  <button type="submit">观察本地提交</button>
</form>
```

`action`指定提交目标，`method`指定提交方法。这里使用GET，提交后浏览器导航到目标页面，字段可以出现在URL的查询部分，例如`pages/received.html?nickname=demo`。本地目标页只是静态说明页，不会把查询内容保存成报名记录。

POST通常把表单数据放在请求体中，但它本身不保证保密，也不等于数据已保存。实际网络过程将在HTTP章节展开。GET会把字段显示在地址中，因此示例只应使用虚构资料。

表单中的按钮应明确类型：`type="submit"`触发提交，`type="button"`是普通按钮，`type="reset"`恢复控件初始值。普通按钮不会凭空获得业务操作；不应因为文字写着“提交”就认为它一定完成提交。表单中的`button`省略类型时，通常具有提交行为。



图2-24 从控件当前值，到提交目标与查询字段

5.7 检查控件能否被实际使用

用Tab键依次访问输入框、选择框和可操作按钮，确认顺序合理。点击标签，检查焦点是否进入对应控件；切换单选项，检查是否保持互斥；尝试空值提交，观察`required`的反馈。

提交字段有问题时，先检查`name`、当前值、选中状态和禁用状态，再考虑后续程序。控件显示正常，不代表它一定会成为提交数据的一部分。



图2-25 不用鼠标，也应能按顺序进入可操作控件

6 图片之外的资源与原生交互

6.1 音频和视频也通过路径引用

媒体元素可以提供浏览器内置的播放控制。以下片段假定项目中已经准备了相应文件：

```
<audio controls>
  <source src="media/introduction.mp3" type="audio/mpeg">
  您的浏览器不支持音频播放。
</audio>
```

```
<video controls poster="images/video-cover.jpg">
  <source src="media/activity.mp4" type="video/mp4">
  <track src="media/captions-zh.vtt" kind="subtitles"
         srclang="zh" label="中文字幕">
  您的浏览器不支持视频播放。
</video>
<p><a href="pages/transcript.html">阅读视频文字稿</a></p>
```

`source`给出候选媒体来源与类型，`controls`显示播放控件，`poster`给出视频封面，`track`关联字幕等时间文本。引用位置仍由当前文档URL与资源路径共同决定。

示例路径只有在真实文件存在时才可播放。元素内部的后备文字主要针对不支持该元素的情况，并不保证在媒体文件找不到时自动成为完整的错误反馈，因此仍应测试资源能否加载，并提供清楚的文字入口。



图2-26 音频与视频，也需要资源地址和可操作控件

6.2 文字替代与字幕服务于内容理解

重要信息不应只存在于声音、视频画面或图片里。音频可以配文字稿，包含口语的视频应提供相应字幕，必要时还应说明画面中无法从声音获得的信息。内容替代与原资源保持一致，才有助于不同条件下的用户获取信息。

不要默认自动播放带声音的内容。提供播放控制，让用户决定何时开始，并检查键盘是否能够使用主要功能。

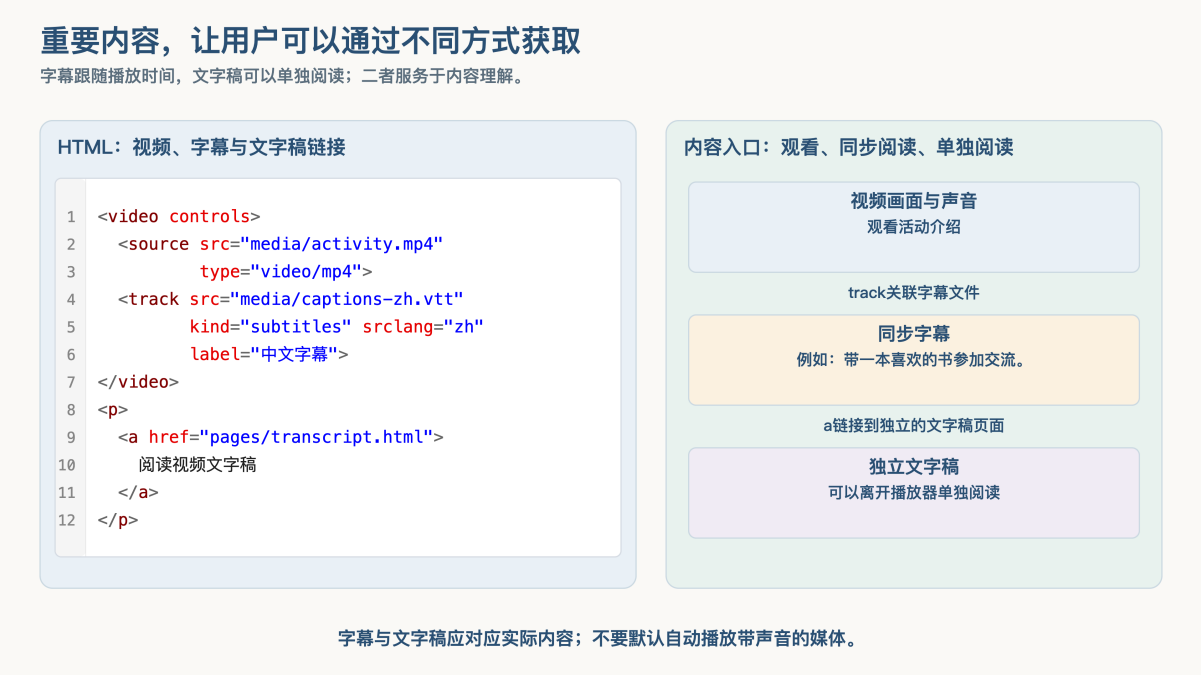


图2-27 重要内容，让用户可以通过不同方式获取

6.3 使用details与summary提供展开内容

```
<details>
  <summary>需要准备什么？</summary>
  <p>带一本自己读过的书，准备三分钟以内的分享。</p>
</details>
```

`summary`提供可操作的概括，`details`组织展开内容。浏览器已经实现展开状态与相应的基本操作，适合活动问答等场景。



图2-28 用details与summary表达展开内容

链接、按钮、输入控件、媒体控件和展开组件都提供了原生能力。使用普通`div`模仿它们时，还需要自行补上键盘、焦点、状态与行为，容易遗漏。能满足需求时，优先使用合适的原生元素。

7 组合成校园活动信息页

7.1 内容组织与目录

完整示例采用“校园读书交流会”，包括活动介绍、图片及图注、日程表、参与提醒和静态表单。目录如下：

```
src/ch02/
├── index.html
├── form-practice.html
├── images/
│   └── activity.svg
└── pages/
    ├── details.html
    └── received.html
```

首页建立主要区域与页内导航，详情页提供跨目录往返链接，本地表单页用于观察标签、必填校验和GET字段。首页没有样式表或脚本依赖，媒体片段属于另外准备资源后可尝试的扩展。



图2-29 一个完整案例，怎样组织页面和本地资源

7.2 从阅读顺序推导元素关系

先确定用户的阅读问题：这是什么活动、何时发生、怎样参加。页头表达活动主题，导航提供入口，主要内容依次回答三个问题。活动介绍中的图片与图注组成`figure`，日程由`table`表达时间关系，参与提醒采用列表。

参与方式部分保留结构完整的静态表单，帮助识别标签、控件和字段。首页按钮为禁用状态，页面不提供真实报名服务。测试提交与校验时，打开单独的`form-practice.html`，使用虚构内容。

完整源码见附录A。其中`activity-card`等类名用于标记同类区域，本身不会产生外观或功能。CH03将在这份结构上添加样式，使内容关系与视觉呈现形成连续的例子。

7.3 从四条线索检查整页

线索	检查内容	具体操作
文档树	区域、标题和嵌套	在Elements中展开main与section
资源关系	图片与跨页链接	从首页进入详情，再返回；核对图片
数据关系	表格标题、标签与字段	找到scope、for、id、name及当前值
原生行为	导航、输入与展开	使用鼠标及键盘逐项操作

页面可以很朴素，但这些关系应成立。文件移动后仍能正常打开，标签能正确聚焦，表格数据能根据标题理解，比临时做出一种外观更能说明HTML结构已经清楚。



图2-30 让内容关系成为明确的HTML关系

7.4 通过有意修改检验理解

把详情页中的返回路径从`../index.html`改成`index.html`，预测浏览器会寻找哪个文件，再改回。把一个标签的`for`改错，检查点击文字的行为。把某个`name`移除，在本地提交页中观察查询字段有何变化。每次只改一处，保留预期与实际结果。

对合并单元格，可以先在纸上画出覆盖位置，再修改属性。不要连续增加空单元格来试出一个“看起来差不多”的表格；这样很难解释网格究竟表达什么。

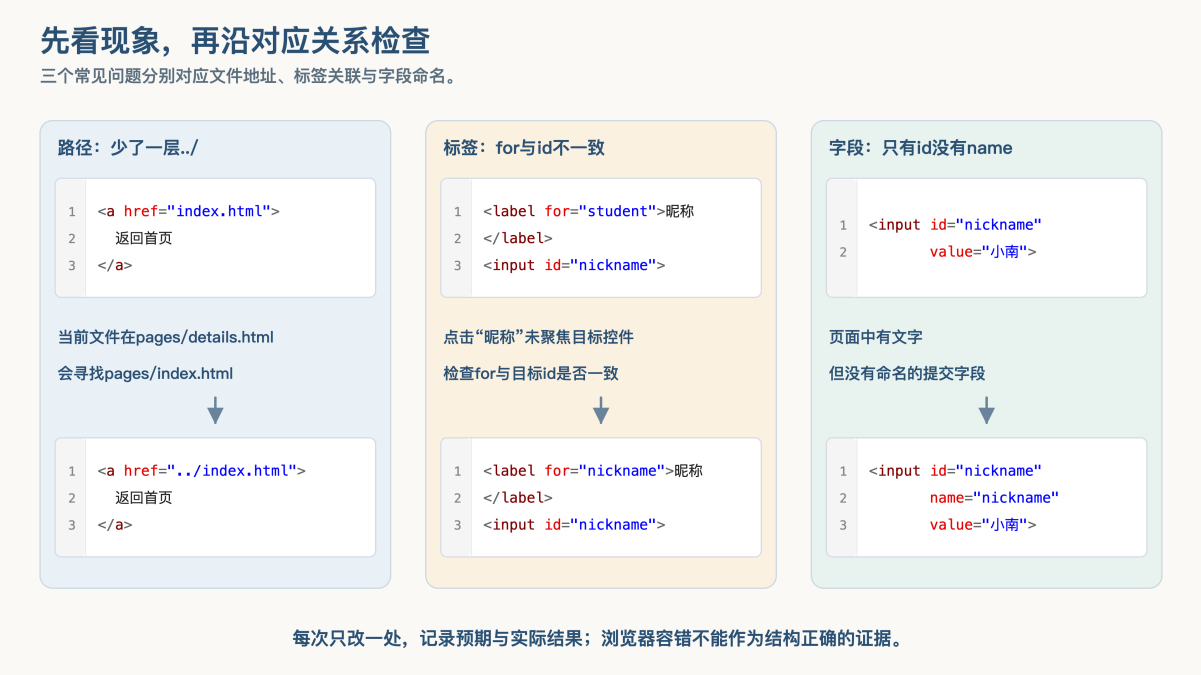


图2-31 先看现象，再沿对应关系检查

本章小结

HTML用元素、属性和嵌套组织文档。浏览器把源代码解析为树，父子、祖先后代与兄弟关系由此建立。合法嵌套需要遵守元素规则，浏览器容错不能作为源代码正确的依据。

语义区域和标题层级组织阅读线索，地址关联文档与资源，替代文本和标签等属性把内容关系明确表达出来。表格通过标题单元格建立行列关系，表单通过标签、控件、名称与值组织输入。合适的原生元素同时提供语义和可操作的浏览器行为。

复习与思考

1. 标签与元素是什么关系？空元素为什么不写结束标签？
2. id、class和name各有什么作用？一个元素能否有多个类？
3. `required="false"`为什么仍可能使控件必填？
4. 父元素与祖先有什么区别？两个元素在源代码中相邻就一定是兄弟吗？
5. 为什么“页面能显示”不能证明嵌套合法？
6. section、article和div应怎样根据内容选择？
7. 从pages/details.html引用同一张图片时，为什么常需要../？
8. alt与figcaption有什么区别？什么时候适合alt=""？
9. 表格一行只有两个单元格元素，为什么仍可能覆盖三个列位置？
10. 标签旁边有文字，是否表示输入框已经有了程序可识别的名称？

11. 控件有id但没有name，通常会怎样影响提交字段？
12. 输入、原生校验、提交与保存记录，是不是同一件事？

常见误解与代码审查

练习一：树形关系与修复

```
<p>活动主题
  <section><h2>阅读与交流</h2></section>
</p>
```

先预测源码希望表达什么，再查看浏览器实际节点。请改为合法且含义清楚的结构。

练习二：从不同目录计算地址

项目结构与第3.1节相同。下面的代码写在`pages/details.html`中，为什么不能正确返回首页？

```
<a href="index.html#schedule">查看活动安排</a>
```

练习三：标签与数据字段

```
<label for="student-name">练习昵称</label>
<input id="nickname" type="text" required="false">
```

指出标签关联、字段命名和布尔属性三个问题，给出一个可以表达“昵称非必填”的合理版本。

练习四：合并后是否还需补列

一张表有三列。最后一行写为：

```
<tr>
  <th scope="row">周六</th>
  <td colspan="2">主题展览</td>
  <td></td>
</tr>
```

请画出覆盖位置，并删除多余单元格。

课后微任务：语义清楚的活动信息页

选择一个虚构校园活动，制作可直接打开的HTML项目。页面包括完整文档骨架、合理的标题与主要区域、至少一组列表、页内导航、详情页往返链接、一张本地图片及相应替代文本、一张带标题关系的日程表，以及有明确标签的静态表单。

使用昵称和虚构资料。若没有提供真实接收服务，应明确页面是练习，并使主页面的提交入口保持不可用。本地提交观察可以独立成页，说明它只演示导航与字段，不产生真实记录。

完成后检查资源路径、重复id、元素嵌套、图片替代文本、表头关系与标签关联；用键盘完成导航和输入。提交项目目录、页面截图及一份验证记录，至少说明一处路径推导、一处父子关系和一处表单字段关系。AI参与时，说明接受的修改以及如何运行核对。

附录A 完整活动首页

配套完整案例位于src/ch02，入口为index.html。以下首页源码与案例文件一致。图片使用本地`images/activity.svg`，详情页与表单练习入口均在项目中提供。

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>校园读书交流会 | HTML结构示例</title>
</head>
<body>
  <header id="top" class="site-header">
    <p class="eyebrow">校园活动 · 教学示例</p>
    <h1 class="page-title">校园读书交流会</h1>
    <p>带一本喜欢的书，分享一次阅读体验。</p>
    <nav class="site-nav" aria-label="页面导航">
      <a href="#introduction">活动介绍</a>
      <a href="#schedule">活动安排</a>
      <a href="#registration">参与方式</a>
    </nav>
  </header>

  <main class="page-content">
```

```
    <section id="introduction" class="activity-card">
      <h2>活动介绍</h2>
      <p class="activity-meta">周五下午 · 图书馆交流空间</p>
      <p>选择一本你喜欢的书，准备一段简短的介绍。你可以分享书中的一个观点，也可以讲述这本书与你的故事。</p>
      <figure>
        <img class="activity-image" src="images/activity.svg"
             alt="展开的书与交流气泡的插画" width="800" height="300">
        <figcaption>从一本书开始，让不同的想法在交流中相遇。</figcaption>
      </figure>
      <p><a href="pages/details.html">阅读活动详情</a></p>
      <a class="action-link" href="#registration">查看参与方式</a>
    </section>

    <section id="schedule" class="activity-card">
      <h2>活动安排</h2>
      <p>所有时间和地点仅用于课堂示例，不代表正式活动安排。</p>
      <table>
        <caption>交流会流程</caption>
        <thead>
          <tr><th scope="col">时间</th><th scope="col">内容</th></tr>
        </thead>
        <tbody>
          <tr><th scope="row">14:00–14:10</th><td>相互认识，介绍带来的书</td></tr>
          <tr><th scope="row">14:10–14:40</th><td>轮流分享阅读体验</td></tr>
          <tr><th scope="row">14:40–15:00</th><td>自由交流与书目推荐</td></tr>
        </tbody>
      </table>
    </section>

    <section class="activity-card notice-card">
      <h2>参与提醒</h2>
      <ul>
        <li>可以选择小说、科普、历史等自己感兴趣的书。</li>
        <li>分享内容控制在三分钟左右。</li>
        <li>尊重不同观点，认真倾听其他同学的介绍。</li>
      </ul>
      <p class="notice-text">请使用昵称，不要在练习页面填写真实联系方式。</p>
```

```
    </section>

    <section id="registration" class="activity-card">
      <h2>参与方式</h2>
      <p id="form-note" class="activity-meta">以下是静态表单练习，不接收、不保存也不发送报名信息。提交按钮暂不可用。</p>
      <form aria-describedby="form-note">
        <div class="form-field">
          <label for="nickname">练习昵称（必填）</label>
          <input id="nickname" name="nickname" type="text" required
                 placeholder="例如：小南">
        </div>
        <div class="form-field">
          <label for="book-type">准备分享的书籍类别</label>
          <select id="book-type" name="bookType">
            <option value="fiction">文学与小说</option>
            <option value="science">科学与技术</option>
            <option value="history">历史与人文</option>
            <option value="other">其他</option>
          </select>
        </div>
        <div class="form-field">
          <label for="book-note">一句话介绍这本书</label>
          <textarea id="book-note" name="bookNote" rows="3"
                    placeholder="写下你最想分享的一点"></textarea>
        </div>
        <button type="submit" disabled>静态练习，暂不提交</button>
      </form>
      <p><a href="form-practice.html">进入本地表单练习</a></p>
    </section>
  </main>

  <footer class="site-footer">
    <p>Web程序设计 · HTML结构与语义</p>
    <a href="#top">返回页首</a>
  </footer>
</body>
</html>
```

附录B 进一步理解内容模型

HTML标准从多个角度描述内容类别，例如流内容、短语内容、标题内容、分区内容、嵌入内容与交互内容。类别可以重叠：一个带href的链接可以同时参与短语内容和交互内容。

这些名称有助于查阅元素规则，但不能把它们理解成一张互斥分类表。判断元素能否放进某个位置，应查阅该元素的内容模型与额外限制。对当前活动页，优先把标题、段落、列表、区域和控件关系写清楚，再在遇到具体问题时查阅标准。

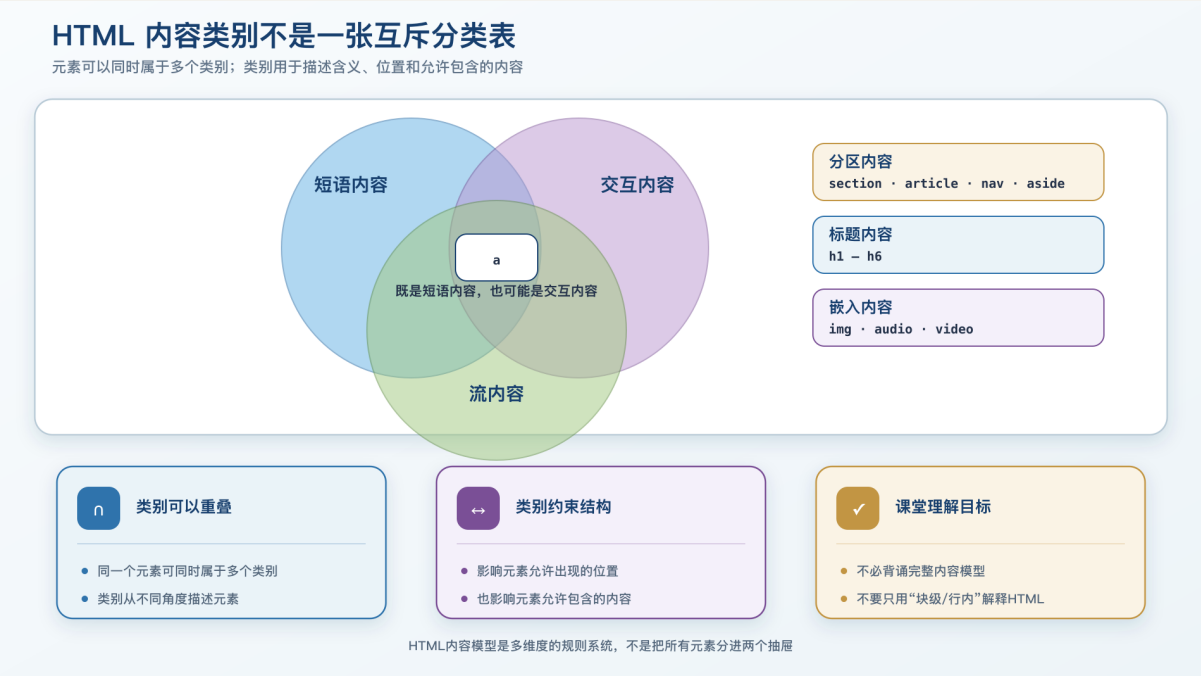


图2-32 HTML内容类别可以重叠

附录C 自测与代码审查参考

C.1 核心问题参考要点

1. 标签是源码标记，元素是结构单位。空元素不包含子内容，也没有结束标签。
2. id唯一标识文档元素，class用于分组，name用于表单字段命名；多个类名用空格分隔。
3. 布尔属性以是否存在表达真假，不能用字符串false关闭。
4. 父元素是直接上一层，祖先包含更上层；兄弟关系要求共享同一父元素。
5. 浏览器会恢复某些错误，源代码与解析结果可能不同。
6. section围绕主题，article强调可独立理解或分发，div用于没有更具体角色的分组。
7. 相对路径从当前文档URL计算，进入pages后需要相应返回上层。
8. alt承担文字替代，figcaption提供可见图注；纯装饰图片可用空alt。

9. colspan或rowspan使一个单元格覆盖多个位置。
10. 视觉相邻不等于label关联，需要for与id一致或合法包含关系。
11. 普通输入框缺少name通常不形成命名字段，id不会自动替代name。
12. 当前值变化、校验、发送与保存分别属于不同阶段。

C.2 代码审查参考

练习一可以把主题段落放到`section`中，避免用`p`包围分区：

```
<section>
  <h2>阅读与交流</h2>
  <p>活动主题：从一本书开始分享。</p>
</section>
```

练习二应写`../index.html#schedule`。原写法会在`pages`下寻找`index.html`。

练习三可改为：

```
<label for="nickname">练习昵称（选填）</label>
<input id="nickname" name="nickname" type="text">
```

练习四的标题占一列、展览占两列，已经覆盖三列，应删除最后一个空`td`。

C.3 资源故障的检查顺序

先确认真实文件存在，再定位引用它的HTML文件，从该文档位置计算路径，核对文件名、扩展名和大小写。路径正确而媒体仍不工作时，再检查文件类型、编码与浏览器支持。路径问题与文件本身能否被解码，是两个不同问题。

参考资料

- WHATWG HTML标准：查阅元素内容模型、解析与表单规则。
- MDN：文档结构：理解语义区域与标题组织。
- MDN：发送表单数据：理解action、method与名称—值关系。
- MDN：disabled属性：理解禁用控件的行为与提交边界。
- MDN：HTML元素参考：查阅图片、表格、表单和媒体元素。